# PRIMEIROS PASSOS

Para iniciar, precisamos verificar se nosso ambiente de trabalho está devidamente configurado.
Vamos verificar a versão do Python que está instalada em nosso computador, para isso, acesse o CMD e insira o comando:

python --version

Caso esteja instalado, aparecerá uma mensagem apresentando a versão instalada:

```
C:\Users\Karen>python --version
Python 3.12.4
```

Caso a versão instalada seja a 3.14 ou superior, recomanda-se que desinstale e instale uma versão anterior, como a 3.12.

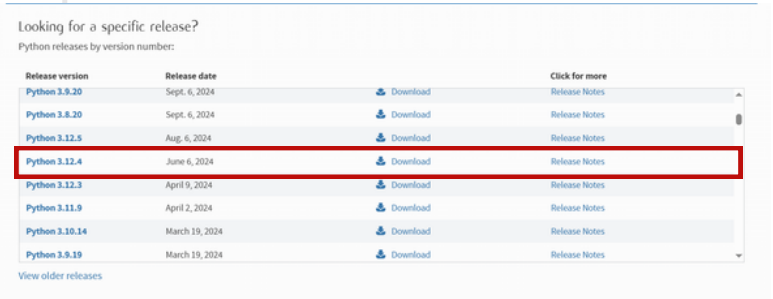
Isso porque o pygame não é uma biblioteca nativa do Python, e atualmente (junho de 2025), ainda não está devidamente adequada a versões mais recentes.

# PRIMEIROS PASSOS

Para instalar o Python, acesse:

https://www.python.org/downloads/

E busque pela versão 3.12.4



Looking for a specific release?

Python releases by version number:

| Release version | Release date | | Click for more |
|---|---|---|---|
| Python 3.9.20 | Sept. 6, 2024 | Download | Release Notes |
| Python 3.8.20 | Sept. 6, 2024 | Download | Release Notes |
| Python 3.12.5 | Aug. 6, 2024 | Download | Release Notes |
| Python 3.12.4 | June 6, 2024 | Download | Release Notes |
| Python 3.12.3 | April 9, 2024 | Download | Release Notes |
| Python 3.11.9 | April 2, 2024 | Download | Release Notes |
| Python 3.10.14 | March 19, 2024 | Download | Release Notes |
| Python 3.9.19 | March 19, 2024 | Download | Release Notes |

View older releases

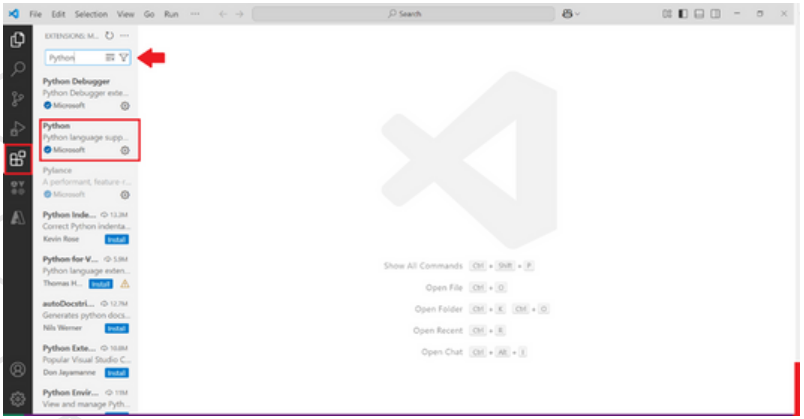
Após realizar o download, faça a instalação. Atente-se para marcar a opção

"Add python.exe to PATH"

Então clique na opção "Install Now" e siga os passos solicitados pelo instalador.

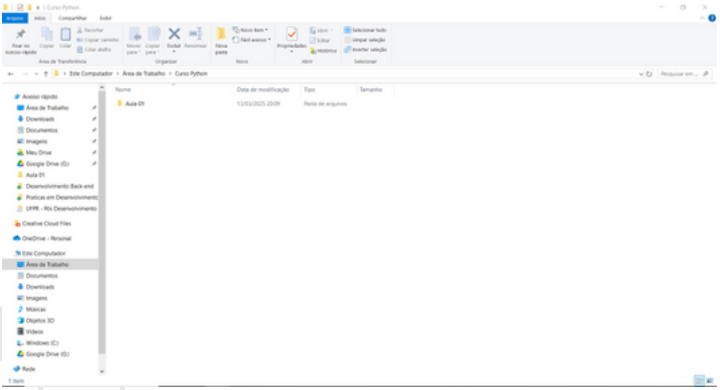
# PRIMEIROS PASSOS

Agora que você já está com o Python instalado em seu computador, abra o VS Code e adicione a extensão Python:

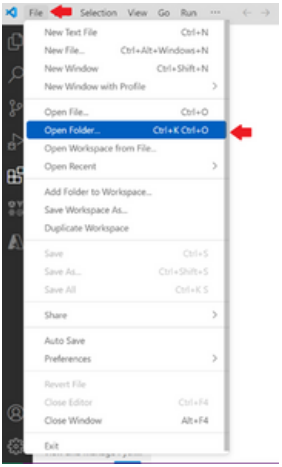



No seu computador, crie uma pasta dedicada aos seus projetos em Python. Essa pasta será o local onde você salvará seus arquivos de código, ajudando a manter tudo organizado. Você pode criar uma pasta em sua Área de Trabalho, em Documentos ou conforme sua organização pessoal. Para facilitar você poderá criar pastas nomeadas como “Aula 01” e “Aula 02” conforme seu progresso.

# PRIMEIROS PASSOS

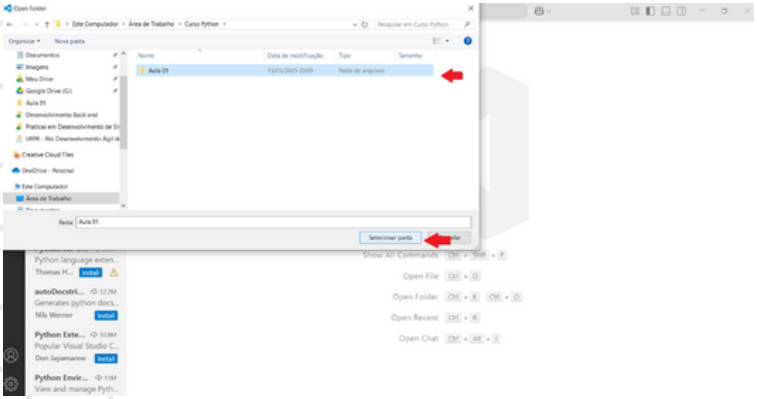


Abra o VS Code e, na opção “Abrir Pasta” (File > Open Folder):

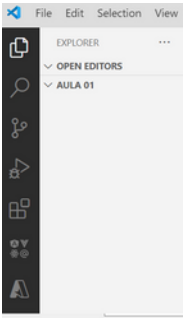


# PRIMEIROS PASSOS

Selecione a pasta que você acabou de criar. Dessa forma, o VS Code tratará essa pasta como um workspace, permitindo acesso rápido aos seus arquivos e facilitando a execução de comandos específicos.

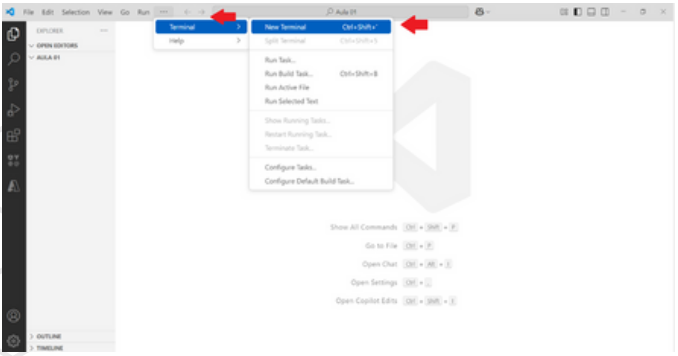


Após abrir sua pasta, ela aparecerá na lateral, onde adicionaremos os arquivos necessários para as aulas:



# PRIMEIROS PASSOS

Uma das vantagens do VS Code é o terminal integrado. Acesse-o através do menu `Terminal > Novo Terminal` ou pelas teclas de atalho Ctrl+Shift+'.




Com o terminal aberto, você pode executar seus scripts Python diretamente do editor, o que agiliza o ciclo de escrita e teste do código.

# PRIMEIROS PASSOS

Explore as configurações do VS Code (ícone de engrenagem no canto inferior esquerdo) para ajustar aspectos como formatação automática do código, atalhos personalizados e preferências de aparência. Essas configurações vão ajudar a criar um ambiente confortável e produtivo para o seu aprendizado.

Investir tempo nessa configuração inicial é fundamental. Ao final dessa etapa, você terá um ambiente de trabalho otimizadíssimo, pronto para transformar suas ideias em código e permitir que você avance com segurança no mundo da programação.

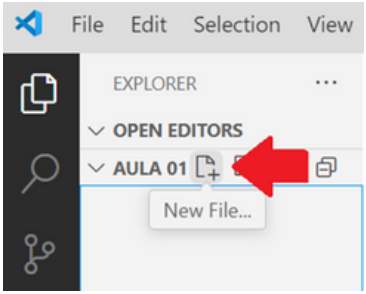
Primeiros Passos em Python

Com o ambiente preparado, é hora de dar os primeiros passos na linguagem Python. Vamos começar com algo simples: você vai aprender a escrever e rodar seu primeiro programa, dando o famoso “Olá, Mundo!” no terminal.

# PRIMEIROS PASSOS

1. Crie um Novo Arquivo:
No VS Code, crie um novo arquivo e salve-o com a extensão `.py` (por exemplo, `primeiro_programa.py`). Essa extensão indica que se trata de um arquivo Python.

Para isso, clique no ícone de Novo Arquivo (New File) logo ao lago do nome da sua pasta Aula 01.




Na sequência, digite “primeiro_programa.py” e pressione a tecla enter no seu teclado. Seu primeiro arquivo será automaticamente aberto.

# PRIMEIROS PASSOS

2. Escreva o Código:
Digite o seguinte comando no arquivo:

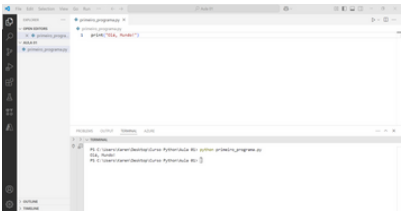
```
print("Olá, Mundo!")
```

Esse simples comando instrui o Python a exibir a mensagem “Olá, Mundo!” no terminal. Apesar de parecer básico, é o seu primeiro contato direto com a escrita e execução de um código.

3. Execute o Programa:
Salve o arquivo pressionando as teclas Ctrl+S em seu teclado e, no terminal integrado do VS Code, digite:

```
python primeiro_programa.py
```

Você deverá ver a mensagem “Olá, Mundo!” impressa na tela. Esse é o sinal de que tudo está funcionando corretamente!



# PRIMEIROS PASSOS

**Sintaxe Básica do Python**

Agora que você já executou seu primeiro programa, é hora de mergulhar um pouco mais fundo na estrutura da linguagem Python. Vamos explorar alguns dos conceitos fundamentais que formarão a base do seu aprendizado:

**1. Variáveis e Tipos de Dados:**
Em Python, você pode atribuir valores a variáveis de forma simples:

```
nome = "João"
idade = 25
```

Aqui, `nome` é uma variável do tipo string e `idade` é um número inteiro. As variáveis são pequenos espaços que reservamos na memória RAM do computador que permitem armazenar informações que podem ser utilizadas e modificadas ao longo do programa.

# PRIMEIROS PASSOS

**2. Operadores Aritméticos:**
Você pode fazer cálculos matemáticos básicos com operadores como +, -, * e /:

```
soma = 10 + 5
produto = 4 * 7
```

Esses operadores são essenciais para a manipulação de valores numéricos e para a criação de lógicas de decisão em seus programas.

**3. Entrada e Saída de Dados:**
Usamos a função `input()` para capturar dados do usuário e `print()` para exibir informações na tela:

```
nome = input("Digite seu nome: ")
print("Bem-vindo(a),", nome)
```

Essa interação permite que seu programa seja dinâmico e responda aos dados fornecidos pelo usuário.

# PRIMEIROS PASSOS

**4. Comentários no Código**

Comentários são trechos de texto que ajudam a documentar o que o código faz e não são executados:

```
# Este é um comentário em Python
```

Usar comentários de forma estratégica facilita o entendimento do código quando você voltar a ele após algum tempo.

Entender e praticar esses conceitos básicos é fundamental para que você se sinta confortável ao escrever códigos mais complexos. Cada linha de código tem um propósito e, à medida que você se familiariza com a sintaxe do Python, verá que a linguagem é bastante intuitiva e direta.

# INICIANDO O JOGO

Vamos iniciar nosso primeiro jogo com Pygame.
Inicie um novo projeto e crie uma pasta para armazená-lo.

Iniciaremos com um modelo básico de um jogo com a biblioteca Pygame. Ele cria uma janela de jogo, dá um título para a janela e mantém ela aberta até o jogador fechá-la. Não tem personagens, nem cenário, mas é a base para começar qualquer jogo.

A primeira linha do nosso código será a importação da biblioteca pygame, da seguinte forma:

```
import pygame as pg
```

Aqui estamos dizendo para o Python usar a biblioteca Pygame, que é usada para criar jogos.
O **as pg** serve para abreviar o nome: em vez de escrever pygame toda hora, podemos usar pg.

Em seguida, importaremos constantes úteis do Pygame. Por exemplo, QUIT, que serve para detectar quando o jogador clica no X para fechar a janela.

```
from pygame.locals import *
```

Para completar a importação dos recursos necessários, importaremos  função exit() da biblioteca sys, que usamos para fechar o programa de forma segura quando o jogador quiser sair.

```
from sys import exit
```

Para inicializar todas as ferramentas do Pygame, usamos o comando:

```
pg.init()
```

É como se estivessemos ligando os motores do jogo antes de começar a usar os recursos gráficos, sons, etc.

Em seguida, iniciaremos a configuração da nossa tela, informando sua dimensão e inicializando-a

Para difinir o da janela do jogo, criaremos as variáveis largura e altura, que receberão os valor referentes a largura e altura da janela

A largura será de 640 pixels e a altura 480 pixels. Imagine como se fosse uma tela do tamanho 640 blocos na horizontal e 480 na vertical.

```
largura = 640
altura = 480
```

O comando:

```
tela = pg.display.set_mode((largura, altura))
```

Cria a janela do jogo com o tamanho que definimos. A função set_mode() prepara essa tela e guarda dentro da variável tela, que podemos usar depois para desenhar coisas nela.

E por fim, adicionamos um título à janela, que será o nome do nosso projeto, você pode adicionar o nome que desejar:

```
pg.display.set_caption("Jogo de Teste")
```

# O LOOP DO JOGO

Imagine que um jogo é como um filme interativo: ele precisa ficar passando várias vezes por quadros (frames), vendo o que está acontecendo, desenhando na tela e respondendo ao jogador.

Para isso, usamos um loop – uma repetição que acontece o tempo todo enquanto o jogo estiver aberto.
Esse loop é chamado de game loop e ele serve para 3 coisas principais, o tempo todo:

**1. Verificar o que o jogador está fazendo**
Dentro do loop, usamos:

```
for evento in pg.event.get():
```

Isso é como perguntar o tempo todo:
“O jogador apertou uma tecla? Clicou no mouse? Tentou fechar o jogo?”

Se não tivesse esse loop, o jogo não saberia quando algo acontece.

**2. Atualizar o que está acontecendo no jogo**
No futuro, quando você tiver personagens se movendo, inimigos aparecendo, pontuação mudando, tudo isso vai acontecer dentro do loop, porque o jogo precisa pensar o tempo todo:
"O que está acontecendo agora? O jogador andou? Um inimigo apareceu?"

**3. Desenhar na tela (atualizar os gráficos)**
Depois de ver o que mudou, o jogo desenha tudo de novo:

```
pg.display.update()
```

Essa linha faz a tela mostrar os novos elementos do jogo (ou seja, o "quadro" atual).
É como pintar um quadro novo várias vezes por segundo – o bastante para parecer uma animação.

Dando sequência ao nosso código, iniciaremos o loop do jogo com um laço infinito:

```
while True:
```

ou seja, uma repetição que nunca para até o jogador fechar o jogo.

Em seguida, verificamos tudo o que está acontecendo, como movimentos do mouse, teclas apertadas e se o jogador clicou no X da janela.

Isso é feito com pg.event.get(), que pega todos os eventos, da seguinte forma:

```
for evento in pg.event.get():
```

Adicionamos uma estrutura condicional para verificar se o tipo do evento for QUIT (ou seja, se o jogador clicou para fechar a janela), então chamamos exit() e o programa termina.

```
if evento.type == QUIT:
    exit()
```

E por fim, atualizamos a janela do jogo. Mesmo que não tenha nada desenhado ainda, essa linha serve para manter a tela funcionando.

```
pg.display.update()
```

Quando colocarmos imagens, personagens e textos, essa função vai garantir que tudo apareça na tela.

Esse código é como o esqueleto de um jogo. Ele ainda não tem cenário, som ou personagens, mas já:

- Abre uma janela com tamanho e nome definidos,
- Escuta eventos do jogador (como fechar a janela)

- E fica rodando para manter o jogo “vivo”.

É o ponto de partida para a gente começar a criar algo divertido!

Revisando o código:

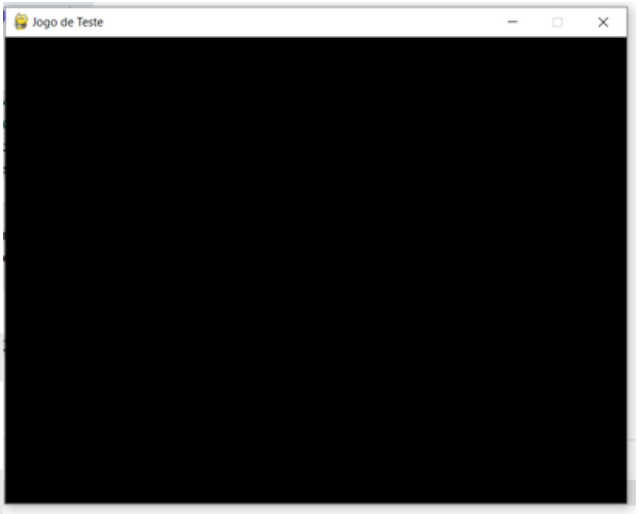
```
import pygame as pg
from pygame.locals import *
from sys import exit

pg.init()

largura = 640
altura = 480
tela = pg.display.set_mode((largura, altura))
pg.display.set_caption("Jogo de Teste")

while True:
  for evento in pg.event.get():
    if evento.type == QUIT:
      exit()

    pg.display.update()
```

Agora já podemos executar a primeira parte do nosso código. No terminal digite:

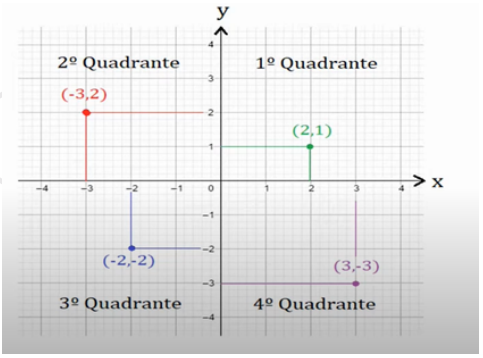
python nome_do_seu_arquivo.py

E dê enter.

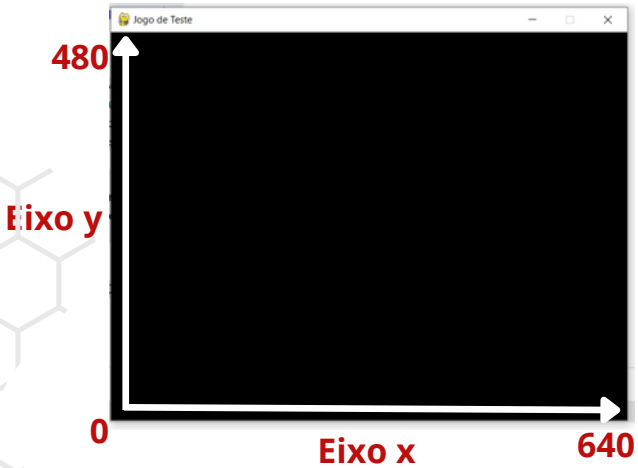
O resultado esperado é a abertura de uma nova janela com o fundo preto e o título do seu jogo:



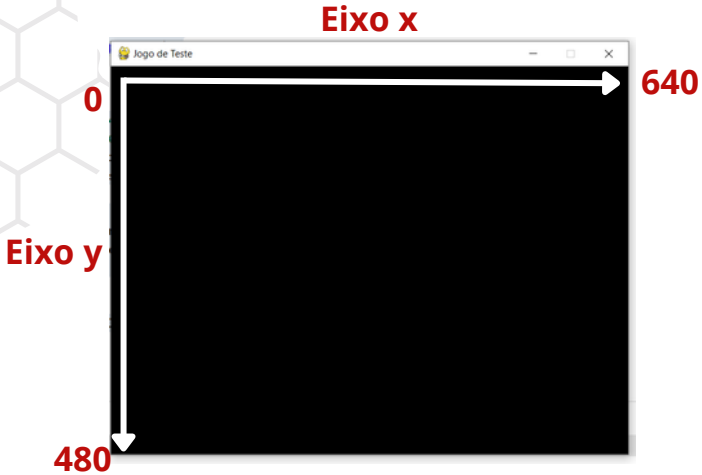

Nosso próximo passo é desenhar algumas formas geométricas. Mas antes, precisamos compreender o plano cartesiano e o sistema de coordenadas para conseguirmos nos localizar.



Anteriormente, definimos a largura e a altura da nossa janela. Isso formará um plano cartesiano quando a janela for inicializada.
Isso significa que teremos um eixo x com 640px de largura, e um eixo y com 480px de altura.
Aprendemos em matemática que o eixo y estará na vertical para cima, da seguinte forma:

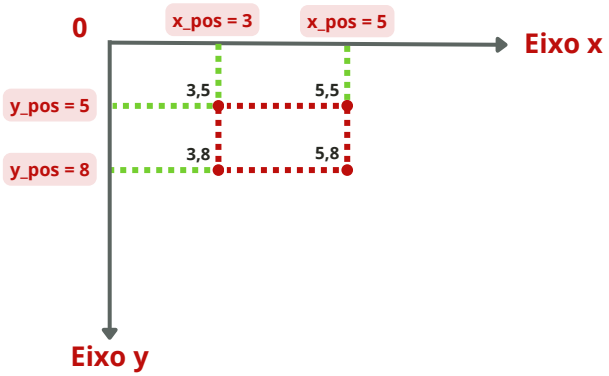



Porém, no pygame, essa lógica se inverte. O eixo y continuará na vertical, porém, para baixo:



Para desenhar qualquer forma, como um retângulo, precisamos indicar as posições no plano x, y da tela. No Pygame, esse plano funciona como uma folha quadriculada, onde:

- O canto superior esquerdo da tela é o ponto (0, 0).
- O eixo x cresce para a direita.
- O eixo y cresce para baixo (diferente da matemática tradicional!).




Na imagem, temos um retângulo formado por 4 vértices com as seguintes coordenadas:

| Vértice | Coordenadas (x, y) |
|---|---|
| Canto superior esquerdo | (3, 5) |
| Canto superior direito | (5, 5) |
| Canto inferior esquerdo | (3, 8) |
| Canto inferior direito | (5, 8) |

Esses pontos definem a posição e o tamanho do retângulo.

Outro ponto a se observar é o sistema de cores utilizado no Pygame, que é baseado no modelo RGB (Red, Green, Blue – Vermelho, Verde e Azul).
Nesse modelo, cada cor é formada pela combinação de três canais de intensidade, que variam de 0 a 255.

Por exemplo:

- A cor preta é representada por (0, 0, 0) – ausência total de luz (nenhuma cor acesa).
- Já a cor branca é representada por (255, 255, 255) – quando todos os canais estão no máximo, somando-se para formar a luz branca.

**Como isso vira código no Pygame?**
Primeiramente vamos definir constantes que armazenarão os valores RGB das cores:

```
VERMELHO = (255, 0, 0)
BRANCO = (255, 255, 255)
```

Para desenhar um retângulo, usaremos a função pygame.draw.rect(), ela necessita de três parâmetros:

**1. tela**
É o local onde o retângulo será desenhado.
No nosso caso, tela é a janela do jogo que criamos com pygame.display.set_mode().

**2. COR**

É a cor do retângulo, definida no padrão RGB (Red, Green, Blue), com números entre 0 e 255.

Você pode usar qualquer cor:

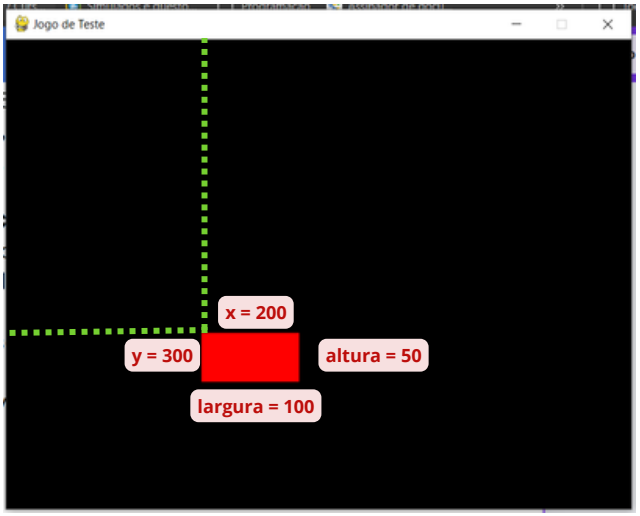
- (0, 0, 0) → Preto
- (255, 255, 255) → Branco
- (0, 255, 0) → Verde
- (0, 0, 255) → Azul
- (255, 255, 0) → Amarelo

3. **POSIÇÃO**

Definimos em qual posição do plano cartesiano vamos desenhar nosso retângulo, informado o ponto x, y e a largura e altura do retângulo.

Em nosso código, ficará da seguinte forma:

```
pg.draw.rect(tela, VERMELHO, (200,300, 100, 50))
```

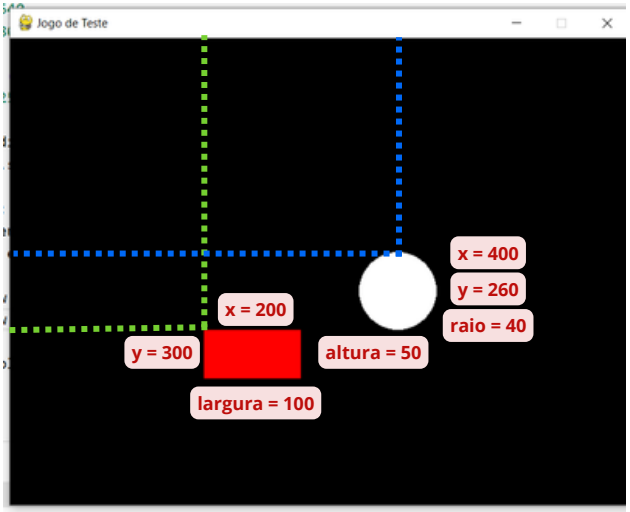


Vamos agora desenhar um círculo. Para isso, usaremos a função:

pygame.draw.circle(tela, cor, posição x e y, raio)

Em nosso código, faremos da seguinte forma:

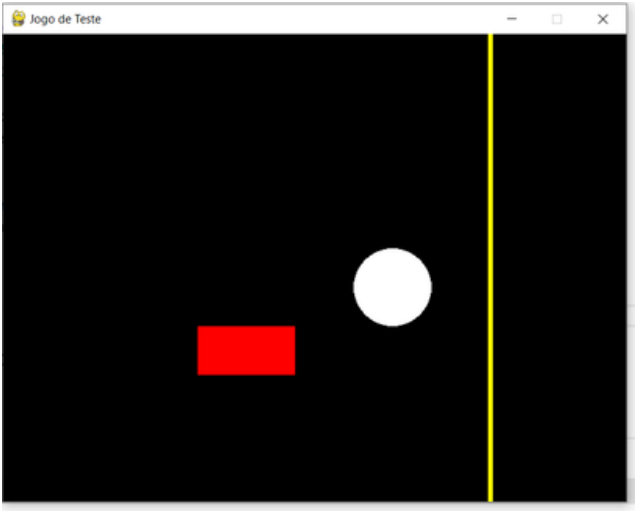
pg.draw.circle(tela, BRANCO, (400,260), 40)




Para completar essa etapa, desenharemos uma linha reta, usando a função:

pygame.draw.linha(tela, cor, posição_x, posição_y, espessura)

Em nosso código ficará:
AMARELO = 255,255,0
pg.draw.linha(tela, AMARELO, (500,0), (500,600),5)




Revisando o código:

```
import pygame as pg
from pygame.locals import *
from sys import exit

pg.init()

largura = 640
altura = 480

VERMELHO = (255, 0, 0)
BRANCO = (255, 255, 255)
AMARELO = (255, 255, 0)

tela = pg.display.set_mode((largura, altura))
pg.display.set_caption("Jogo de Teste")

while True:
    for evento in pg.event.get():
        if evento.type == QUIT:
            exit()
    pg.draw.rect(tela, VERMELHO, (200,300, 100, 50))
    pg.draw.circle(tela, BRANCO, (400,260), 40)
    pg.draw.line(tela, AMARELO, (500,0), (500,600),5)
    pg.display.update()
```

**Adicionando movimento aos objetos:**

Vamos apagar as outras formas geométricas (como círculo e linha) que havíamos criado, e deixar apenas um retângulo vermelho.
Antes, colocávamos a posição do retângulo diretamente dentro da função, como:

```
pg.draw.rect(tela, VERMELHO, (200, 300, 40, 50))
```

Agora, vamos usar variáveis para controlar a posição, o que nos permite movimentar o retângulo:
x = 0
y = 0
Essas duas variáveis definem a posição inicial do retângulo na tela:

- x = 0 → começa no canto esquerdo
- y = 0 → começa no topo

Com as variáveis criadas, vamos substituir os valores fixos por elas:

```
pg.draw.rect(tela, VERMELHO, (x, y, 40, 50))
```

Agora, o retângulo será desenhado na posição (x, y) e terá largura 40 e altura 50.

Usar variáveis para a posição nos permite alterar seus valores dentro do loop do jogo, criando a sensação de movimento!

**Adicionando movimento aos objetos:**

Vamos apagar as outras formas geométricas (como círculo e linha) que havíamos criado, e deixar apenas um retângulo vermelho.
Antes, colocávamos a posição do retângulo diretamente dentro da função, como:

```
pg.draw.rect(tela, VERMELHO, (200, 300, 40, 50))
```

Agora, vamos usar variáveis para controlar a posição, o que nos permite movimentar o retângulo:
x = 0
y = 0
Essas duas variáveis definem a posição inicial do retângulo na tela:

- x = 0 → começa no canto esquerdo
- y = 0 → começa no topo

Com as variáveis criadas, vamos substituir os valores fixos por elas:

```
pg.draw.rect(tela, VERMELHO, (x, y, 40, 50))
```

Agora, o retângulo será desenhado na posição (x, y) e terá largura 40 e altura 50.

Usar variáveis para a posição nos permite alterar seus valores dentro do loop do jogo, criando a sensação de movimento!

Para que o retângulo possa se mover para baixo, criando a ilusão de que está "caindo" na tela, precisamos alterar o valor da variável y de forma automática. Dentro do loop principal do jogo, colocamos:

```
y = y + 1
```

Isso faz com que o valor de y aumente a cada frame, ou seja:

- No primeiro frame: y = 0
- No segundo frame: y = 1
- No terceiro: y = 2
- ... e assim por diante.

Como y representa a posição vertical, esse aumento faz o retângulo descer na tela.
Se não controlarmos a velocidade, o retângulo pode cair rápido demais (depende do processador do computador). Para isso, usamos o recurso de tempo do Pygame.
Logo após inicializar o Pygame, crie o relógio:

```
relogio = pg.time.Clock()
```

E dentro do loop, logo no início, adicione:

```
relogio.tick(30)
```

Isso diz:
“Atualize o jogo no máximo 30 vezes por segundo (30 FPS)”.

Quando o retângulo chegar no fim da tela (parte de baixo), ele deve reiniciar lá em cima para simular um movimento contínuo.

Sabemos que a altura da tela é guardada na variável altura. Então, podemos criar uma condição para verificar:

```
if y >= altura:
    y = 0
```

Isso significa: “Se a posição vertical do retângulo for igual ou maior que a altura da janela, volte ele para o topo.”
Para fazer o retângulo se mover tanto no eixo y (vertical) quanto no eixo x (horizontal), criando um movimento diagonal, é preciso inicializar x no centro.
A posição x vai começar no meio da tela:

x = largura / 2

Como vimos antes, isso posiciona o retângulo no centro horizontal da janela.
Porém, você perceberá que ele não ficará exatamente no centro, isso porque o ponto (x, y) não representa o centro do retângulo, e sim o canto superior esquerdo dele. O retângulo vai ficar deslocado um pouco para a direita, porque ele começa a ser desenhado no meio, e seu corpo se estende a partir dali.

Para centralizar corretamente, você precisa ajustar x e y levando em conta o tamanho do retângulo:

largura_ret = 40
altura_ret = 50

x = (largura / 2) - (largura_ret / 2)
y = (altura / 2) - (altura_ret / 2)

Assim, o centro do retângulo vai bater com o centro da tela.

**Controlando a velocidade do movimento**

Controlamos a taxa de atualização do jogo com Clock, que define quantas vezes por segundo a tela do jogo será atualizada. Isso é feito com:

relogio.tick(FPS)

O FPS (frames por segundo) controla quantas “fotos” por segundo o jogo desenha.

| FPS | Efeito Visual |
|---|---|
| 10 | Muito lento, "travado' |
| 30 | Médio, ideal para test |
| 60 | Suave e fluido |
| 100 | Muito rápido |

No código, faremos a seguinte alteração:

relogio = pg.time.Clock()

while True:
    relogio.tick(30)  # controla para 30 quadros por segundo

Revisando o código:

```
import pygame as pg
from pygame.locals import *
from sys import exit

pg.init()

largura = 640
altura = 480
x = largura/2
y = 0
relogio = pg.time.Clock()

VERMELHO = (255, 0, 0)
BRANCO = (255, 255, 255)
AMARELO = (255, 255, 0)
PRETO = (0, 0, 0)

tela = pg.display.set_mode((largura, altura))
pg.display.set_caption("Jogo de Teste")

while True:
    relogio.tick(100)
    tela.fill(PRETO)
    for evento in pg.event.get():
        if evento.type == QUIT:
            exit()
    pg.draw.rect(tela, VERMELHO, (x, y, 40, 50))
    if y >= altura:
        y = 0
    y = y+ 1

    pg.display.update()
```

**Controlando objetos:**
Vamos permitir que o jogador mova o retângulo manualmente usando as teclas:

- ⬆️ Cima (↑)
- ⬇️ Baixo (↓)
- ⬅️ Esquerda (←)
- ➡️ Direita (→)

Você pode substituir por WASD, ou qualquer outa que desejar.
Usamos a função:

pg.key.get_pressed()

Ela retorna uma lista com o estado de todas as teclas:

- Se uma tecla está pressionada, ela vale True.
- Se não está pressionada, vale False.

Depois, usamos as teclas específicas com os nomes:

K_UP, K_DOWN, K_LEFT, K_RIGHT

(importados automaticamente com from pygame.locals import *)

Código para verificar e mover:

```
if pg.key.get_pressed()[K_UP]:
   y -= 20if pg.key.get_pressed()[K_DOWN]:
   y += 20if pg.key.get_pressed()[K_LEFT]:
   x -= 20if pg.key.get_pressed()[K_RIGHT]:
   x += 20
```

Cada tecla move o retângulo 20 pixels por vez:

- y -= 20 → sobe
- y += 20 → desce
- x -= 20 → vai para a esquerda
- x += 20 → vai para a direita
-

Onde isso deve ser colocado?

Dentro do loop while True:, antes de pg.draw.rect(...), assim:

```
while True:
    relogio.tick(30)

    for evento in pg.event.get():
        if evento.type == QUIT:
            exit()

# Checar teclas pressionadas

if pg.key.get_pressed()[K_UP]:
        y -= 20if pg.key.get_pressed()[K_DOWN]:
        y += 20if pg.key.get_pressed()[K_LEFT]:
        x -= 20if pg.key.get_pressed()[K_RIGHT]:
        x += 20

    tela.fill(PRETO)
    pg.draw.rect(tela, VERMELHO, (x, y, 40, 50))
    pg.display.update()
```

Revisando o código:

```
import pygame as pg
from pygame.locals import *
from sys import exit

pg.init()

largura = 640
altura = 480
x = largura/2
y = altura/2
relogio = pg.time.Clock()

VERMELHO = (255, 0, 0)
BRANCO = (255, 255, 255)
AMARELO = (255, 255, 0)
PRETO = (0, 0, 0)

tela = pg.display.set_mode((largura, altura))
pg.display.set_caption("Jogo de Teste")

while True:
    relogio.tick(60)
    tela.fill(PRETO)
    for evento in pg.event.get():
        if evento.type == QUIT:
            exit()
    if pg.key.get_pressed()[K_UP]:
        y -= 20
    if pg.key.get_pressed()[K_DOWN]:
        y += 20
    if pg.key.get_pressed()[K_LEFT]:
        x -= 20
    if pg.key.get_pressed()[K_RIGHT]:
        x += 20



    pg.draw.rect(tela, VERMELHO, (x, y, 40, 50))

    pg.display.update()
```

**Colisões:**

Nosso objetivo agora é detectar quando o retângulo vermelho (controlado pelo jogador) encosta no retângulo amarelo (fixo), e executar uma ação quando isso acontece.

Utilizaremos o seguinte código:

```
ret_vermelho = pg.draw.rect(tela, VERMELHO, (x, y, 40, 50))
ret_amarelo = pg.draw.rect(tela, AMARELO, (200, 300, 40, 50))

if ret_vermelho.colliderect(ret_amarelo):
    print("Colisão detectada!")
```

O que está acontecendo aqui?

1. Criando os dois retângulos
   - ret_vermelho: desenhado com base nas variáveis x e y (movimento).
   - ret_amarelo: fixo na posição (200, 300).

Importante: a função pg.draw.rect() retorna um objeto Rect, que guarda as coordenadas e dimensões do retângulo.
Por isso, podemos guardar em variáveis (ret_vermelho, ret_amarelo).

2. Detectando colisão
A função:

```
ret_vermelho.colliderect(ret_amarelo)
```

Retorna True se os dois retângulos encostarem ou se sobrepuserem.

3. Resultado da colisão
Se os retângulos colidirem, o programa imprime:

```
print("Colisão detectada!")
```

Aqui você pode futuramente:

- Exibir uma mensagem na tela
- Tocar um som
- Finalizar o jogo
- Contar pontos, etc.

**Colisões e reposicionamento aleatório:**

Quando o jogador (retângulo vermelho) encostar no retângulo amarelo (o “alvo” ou “item”), esse alvo deve se mover para outro lugar aleatório da tela, simulando uma coleta.

Inicialmente, importaremos a função randint() da biblioteca random, que retorna um número aleatório entre a e b.

Ele servirá para sortear a posição inicial do retângulo amarelo e reposicionar o alvo após uma colisão.

Vamos então declarar duas novas variáveis para armazenar a posição aleatório do retângulo amarelo, usando o randint:

```
x_amarelo = randint(0, largura)
y_amarelo = randint(0, altura)
```

Atualizaremos também os parâmetros do retângulo amarelo, para que ele receba as posições aleatórias:

```
ret_amarelo = pg.draw.rect(tela, AMARELO, (x_amarelo, y_amarelo, 40, 50))
```

Faremos uma verificação de colisão, para que se o retângulo vermelho encostar no retângulo amarelo, mova o amarelo para uma nova posição aleatória na tela:

```
if ret_vermelho.colliderect(ret_amarelo):
    x_amarelo = randint(0, largura - 40)
    y_amarelo = randint(0, altura - 50)
```

Revisando o código:

```
import pygame as pg
from pygame.locals import *
from sys import exit
from random import randint

pg.init()

largura = 640
altura = 480
x = largura/2
y = altura/2

x_amarelo = randint(0, largura)
y_amarelo = randint(0, altura)

VERMELHO = (255, 0, 0)
BRANCO = (255, 255, 255)
AMARELO = (255, 255, 0)
PRETO = (0, 0, 0)

tela = pg.display.set_mode((largura, altura))
pg.display.set_caption("Jogo de Teste")
relogio = pg.time.Clock()


while True:
    relogio.tick(60)
    tela.fill(PRETO)
    for evento in pg.event.get():
        if evento.type == QUIT:
            exit()
    if pg.key.get_pressed()[K_UP]:
        y -= 20
    if pg.key.get_pressed()[K_DOWN]:
        y += 20
    if pg.key.get_pressed()[K_LEFT]:
        x -= 20
    if pg.key.get_pressed()[K_RIGHT]:
        x += 20
    ret_vermelho = pg.draw.rect(tela, VERMELHO, (x, y, 40, 50))
    ret_amarelo = pg.draw.rect(tela, AMARELO, (x_amarelo, y_amarelo, 40, 50))

    if ret_vermelho.colliderect(ret_amarelo):
        x_amarelo = randint(0, largura - 40)
        y_amarelo = randint(0, altura - 50)
    pg.display.update()
```

**Pontuação:**

Agora que já temos um sistema de colisão e atuailização de posições, vamos adicionar a pontuação.
Contaremos quantas vezes o jogador colide com o retângulo amarelo e mostraremos na tela.
Primeiro, criaremos a fonte e a variável de pontos

```
fonte = pg.font.SysFont("arial", 20, True, True)
pontos = 0
```

- SysFont("arial", 20, True, True) cria uma fonte Arial tamanho 20, em negrito e itálico.
- pontos = 0 é a variável que guarda a pontuação do jogador, que começa em zero.

Em seguida, criamos a mensagem de pontuação dentro do loop

```
mensagem = f'Pontos: {pontos}'
texto_formatado = fonte.render(mensagem, True, BRANCO)
tela.blit(texto_formatado, (10, 10))
```

Isso cria um texto dinâmico que muda conforme os pontos aumentam.

render() transforma a string em uma imagem que pode ser desenhada na tela.

blit() exibe esse texto na tela na posição (10, 10) (canto superior esquerdo).

Para atualizar a pontuação ao colidir, usamos o seguinte comando:

```
if ret_vermelho.colliderect(ret_amarelo):
 x_amarelo = randint(0, largura - 40)
 y_amarelo = randint(0, altura - 50)
 pontos += 1
```

Quando o jogador encosta no retângulo amarelo:

O retângulo amarelo se move para outro lugar.
A variável pontos aumenta em 1

**Trilha sonora:**

Vamos agora adicionar uma trilha sonora e sons de animação em nosso jogo.
Utilizaremos a função mixer, do pygame.

Precisamos primeiramente buscar sons para adicionar ao nosso jogo. Dois sites que podem ser utilizados são:

https://freemusicarchive.org/
https://themushroomkingdom.net/media/smw/wav

Busque os sons que desejar e baixe-os.

Em seguida, salve-os na pasta do seu projeto para que possamos utilizá-los.

Agora adicionaremos esses sons às respectivas variáveis. Encontre um para ser a música de fundo e outro para as colisões.
Começaremos com a música de fundo, adicionando à variável:

```
musica_fundo = pg.mixer.music.load('musica_fundo.mp3')
```

Adicionamos um looping infinito para que ela sempre fique tocando:

```
pg.mixer.music.play(-1)
```

E também podemos alterar seu volume:

```
pg.mixer.music.set_volume(0.5)
```

Em seguida, vamos adicionar o som de colisão:

```
barulho_colisao = pg.mixer.Sound('smw_coin.wav')
```

E adicionamos a lógica para que ele seja tocado cada vez que houver uma colisão:

```
if ret_vermelho.colliderect(ret_amarelo):
    ...
    barulho_colisao.play()
```

Desta forma, quando o jogador encosta no retângulo amarelo, além de ganhar um ponto e mover o item, o jogo toca o som carregado.

Revisando o código:

```python
import pygame as pg
from pygame.locals import *
from sys import exit
from random import randint

pg.init()

pg.mixer.music.load('musica_fundo.mp3')
pg.mixer.music.play(-1)
pg.mixer.music.set_volume(0.5)
barulho_colisao = pg.mixer.Sound('smw_coin.wav')

largura = 640
altura = 480
x = largura/2
y = altura/2

x_amarelo = randint(0, largura)
y_amarelo = randint(0, altura)

fonte = pg.font.SysFont("arial", 20, True, True)
pontos = 0

VERMELHO = (255, 0, 0)
BRANCO = (255, 255, 255)
AMARELO = (255, 255, 0)
PRETO = (0, 0, 0)

tela = pg.display.set_mode((largura, altura))
pg.display.set_caption("Jogo de Teste")
relogio = pg.time.Clock()


while True:
    relogio.tick(60)
    tela.fill(PRETO)
    mensagem = f'Pontos: {pontos}'
    texto_formatado = fonte.render(mensagem, True, BRANCO)
    tela.blit(texto_formatado, (10, 10))
    for evento in pg.event.get():
        if evento.type == QUIT:
            exit()
    if pg.key.get_pressed()[K_UP]:
        y -= 20
    if pg.key.get_pressed()[K_DOWN]:
        y += 20
    if pg.key.get_pressed()[K_LEFT]:
        x -= 20
    if pg.key.get_pressed()[K_RIGHT]:
        x += 20
    ret_vermelho = pg.draw.rect(tela, VERMELHO, (x, y, 40, 50))
    ret_amarelo = pg.draw.rect(tela, AMARELO, (x_amarelo, y_amarelo, 40, 50))

    if ret_vermelho.colliderect(ret_amarelo):
        x_amarelo = randint(0, largura - 40)
        y_amarelo = randint(0, altura - 50)
        pontos += 1
        barulho_colisao.play()
    pg.display.update()
```

🕹️ Sua vez!

Agora que você já conhece as principais funções para criar a mecânica de um jogo, é hora de deixar tudo ainda mais interessante.

Adicione limites de colisão para evitar que o retângulo vermelho saia da tela – isso tornará o jogo mais desafiador e realista!